AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

24AIM332 · 50% OF THE COURSE

Class Assignment: Ship One System End to End

One application, containerised, orchestrated, declared as infrastructure code and delivered by a pipeline. Built across the semester, submitted once.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

COURSE	24AIM332 Introduction to Cloud Computing
WEIGHT	50% of the course
MARKS	50
WORK	Individual
SUBMIT	A repository plus a report
OFFERING	Odd Semester 2026-27

What this is

The whole assessed practical component of this course is **one deliverable**: a small application that you containerise, orchestrate, provision from code, and deliver through a pipeline.

It is not eight separate submissions. It is one system, built up in stages, and **each practice lab builds exactly one stage**. If you do the labs as they come, this assignment is nearly finished by week 14. If you skip them, you are learning four unfamiliar technologies at once, against a deadline, with a live cloud account and a real bill.

> **NOTE · START THE REPOSITORY IN WEEK 3**
>
> Create one repository now and keep every lab's work in it. That repository *is* the submission. Nothing you do in a lab is thrown away.

The application

Anything small with a build step and a dependency. A Flask or FastAPI service, an Express app, or a static site with a build. It does not have to be original or impressive: the assessment is about **how you ship it**, not what it does.

It must expose an HTTP endpoint and at least one endpoint suitable for a health check.

Stage 1: Containerise it (10 marks)

REQUIREMENT	MARKS
Builds and runs, reachable on a mapped port	2
Multi-stage build: compile or install in one image, copy artefacts into a slim runtime	3
Dependencies installed **before** source is copied, so the layer cache works	2
Base image pinned to a version, not `latest`	1
Runs as a **non-root** user	1
`.dockerignore`, and no secret baked into any layer	1

Report the image size for a naive single-stage build and for your final one, and the rebuild time after a one-line source change, before and after fixing the layer order.

PITFALL · A BAKED SECRET IS RECOVERABLE

Anything placed in an `ENV` or `ARG` line survives in the image history and can be read back with `docker history --no-trunc`, even if a later layer deletes it. Secrets are injected at **run** time. Lab 3 has you prove this to yourself.

Stage 2: Orchestrate it (12 marks)

REQUIREMENT	MARKS
A `Deployment` with 3 replicas and a selector that matches its own pod labels	3
A `Service` routing to those pods, with non-empty endpoints demonstrated	2
Liveness and readiness probes on different paths, with the difference explained	3
`requests` **and** `limits` for cpu and memory	2
A rolling update demonstrated, and a deliberately broken image shown to stall the rollout rather than take the service down	2

A local `kind` cluster is fine and costs nothing. Use a managed cluster only if you specifically want to show a cloud load balancer.

PITFALL · THE TWO MOST COMMON ERRORS, BOTH SILENT

A selector that does not match the pod template labels means the Deployment manages **no pods** and reports no error. Missing `requests` means the scheduler assumes the pod needs nothing, packs the node, and degrades everything on it. Both are checked.

Stage 3: Declare it as code (12 marks)

REQUIREMENT	MARKS
At least three related resources declared in OpenTofu or Terraform	3
`plan` read and reported before every `apply`	2
Idempotence proved: a second `plan` immediately after `apply` reports no changes	3
Drift detected: change something by hand, show the tool notices and says what it will do	2
Environments differ by a variables file, not by duplicated code	2

DEFINITION · WHY IDEMPOTENCE IS THE PROPERTY THAT MATTERS

If applying twice changes something the second time, the configuration never converges, and every apply looks successful while the infrastructure quietly disagrees with its description. You discover this on the day you need the description to be true.

Stage 4: Deliver it with a pipeline (12 marks)

REQUIREMENT	MARKS
Every push runs lint and tests, with the fast checks first	3
An image is built and tagged with the **commit SHA**, not `latest`	2
The **same artefact** is deployed, with the digest shown to match	3
Secrets injected from the CI secret store, and shown not to appear in logs	2
Deployment gated: only from the default branch, only after tests pass, demonstrated with a failing pull request	2

Report the time-to-failure for a lint error before and after ordering your stages fast-first.

PITFALL · BUILD ONCE, PROMOTE

Rebuilding the image separately for each environment means the thing you tested is not the thing you shipped. Build one artefact, and move that same digest through each environment.

Stage 5: Report and teardown proof (4 marks)

REQUIREMENT	MARKS
A short report: what you built, the decisions you made and why	2
Cost: what you actually spent, and what you did to keep it near zero	1
Teardown proof: CLI output showing every resource is gone	1

PITFALL · SCREENSHOTS ARE NOT TEARDOWN PROOF

Paste the output of the list command for each resource type, showing nothing left. A console screenshot proves what one page looked like at one moment.

Check specifically for the things that survive deleting an instance: disks, static IP addresses, load balancers, managed databases, and snapshots.

What to submit

1. **A repository**, public or shared with me, containing your Dockerfile, manifests, infrastructure code, pipeline definition and application.
2. **A report**, `icc-assignment-<ROLLNO>.pdf`, with the measurements each stage asks for, the required evidence, and your teardown proof.

The deadline and submission channel are announced in class.

How this maps to the labs

STAGE	BUILT IN
1, containerise	Labs 3 and 4
2, orchestrate	Labs 5 and 6
3, infrastructure as code	Lab 7
4, pipeline	Lab 8
5, teardown proof	Lab 1, and every lab after it

Policies

Late. 80 percent within 48 hours, then not accepted without documented cause.

Collaboration. Discuss freely. Submit only work you produced. Your repository history is part of the evidence: a single commit the night before the deadline invites questions.

AI tools. Permitted for learning. You must be able to explain every line of every manifest, template and pipeline you submit. Cloud configuration produced without understanding is how expensive and insecure systems get built, so this matters more here than in most courses.

Secrets. A submission containing a live credential scores **zero** and you rotate the key immediately. This is not a penalty for carelessness; it is the same rule that applies in any professional setting.